Practice · Array loops

Each question lists its options; the right one is marked **Right answer**. *Feedback* is what students see after they answer. In a typed question the blank is in square brackets, with the accepted answers separated by `|`.

(mc) L1 · The array loop

`int a[8];` has been made. Which loop visits every box once, from the first to the last?

- `for (int i = 1; i <= 8; i++)`
- `for (int i = 0; i <= 8; i++)`
- **Right answer:** `for (int i = 0; i < 8; i++)`
- `for (int i = 1; i < 8; i++)`

Feedback: The boxes are `a[0]` to `a[7]`. The loop must start at 0 and stop **before** 8: `i < 8`. With `i <= 8` it also reads `a[8]`, which is not in the array; starting at 1 misses `a[0]`. Class 9, slides 37 to 42.

(mc) L1 · Reading into a box

Inside the loop `for (int i = 0; i < 5; i++)`, which line reads a whole number into box `i` of `int a[5];`?

- **Right answer:** `scanf("%d", &a[i]);`
- `scanf("%d", a[i]);`
- `scanf("%d", &a);`
- `scanf("%d", &i);`

Feedback: `&a[i]` means: put the number into box `i`. The `&` is needed, as with `&x`. `&i` would overwrite the loop counter. Class 9, slide 48.

(mc) L1 · Why #define N

Why do we write `#define N 5` at the top and then `int a[N];` and `i < N`?

- Because an array size must always be a capital letter.
- Because `#define` makes the program run faster.
- **Right answer:** So the size is written in one place: to change it, you change one line.
- Because `int a[5];` does not compile.

Feedback: Without `N`, the 5 is in the array and in every loop. Change one and forget another, and you have a bug. With `#define N 5` there is one place to change. Class 9, slides 57 and 58.

(mc) L2 · The running total

```
int e[5] = {4, 1, 7, 3, 9};
int s = 0;

for (int i = 0; i < 5; i++) {
    s += e[i];
}
printf("%d\n", s);
```

What does it print?

- `9`

- `20`
- `10`
- **Right answer:** `24`

Feedback: `s` starts at 0 and gets every box added: 4, 5, 12, 15, 24. If you said 10, you added the indexes 0 + 1 + 2 + 3 + 4, not the values. Class 9, slides 50 to 52.

(mc) L2 · Every second box

```
int k[5] = {3, 6, 9, 12, 15};

for (int i = 0; i < 5; i += 2) {
    printf("%d ", k[i]);
}
```

What does it print?

- `3 6 9`
- **Right answer:** `3 9 15`
- `6 12`
- `3 9 15` and then a number that is not in the array

Feedback: `i += 2` makes `i` go 0, 2, 4, and then 6, where `6 < 5` is false and the loop stops. So it prints `k[0]`, `k[2]` and `k[4]`: 3, 9, 15. Class 9, slides 37 to 41.

(mc) L2 · Change every box

```
int g[5] = {1, 2, 3, 4, 5};

for (int i = 0; i < 5; i++) {
    g[i] = g[i] * 10;
}
printf("%d\n", g[2]);
```

What does it print?

- `20`
- `3`
- `2`
- **Right answer:** `30`

Feedback: The loop multiplies every box by 10, so the array becomes 10, 20, 30, 40, 50. `g[2]` is the third box: 30. Class 9, slides 74 and 75.

(fib) L3 · Write the array loop

Write the first line of the loop that visits every box of an array of `N` boxes, from the first to the last. Type it exactly as on the slides: `int i`, one space after each `;`, and spaces around `=` and `<`.

[for (int i = 0; i < N; i++)]

Feedback: Start at 0, the first box. Stop before `N`, because the last box is number N-1. Add 1 each time. Class 9, slide 42.

(fib) L3 · Write the backwards loop

Write the first line of the loop that visits every box of an array of `N` boxes **backwards**, from the last to the first. Type it exactly as on the slides, with spaces around `=`, `-` and `>=`.

[for (int i = N - 1; i >= 0; i--)]

Feedback: The last box is number `N - 1`. Go down with `i--`, and keep going while `i >= 0`, so box 0 is visited too. With `i > 0` the first box would be missed. Class 9, slides 53 to 56.

(fib) L3 · Write the #define line

Write the line that makes `N` mean 20 in the whole program. Type it exactly, with one space between the parts and nothing at the end.

[#define N 20]

Feedback: `#define N 20`: no `=` and no `;`. Everywhere below this line, gcc reads `N` as 20. Class 9, slides 57 and 58.